

OMI---IMO as Canonical Chiral URI-Compatible Memory Notation

A Dialogic Explainer on `omi---imo`, `o---o/---/?---?@---@`, ASCII Chirality, FS/GS/RS/US, and Bidi Projection

Abstract

This paper explains why the notation:

```
omi---imo
```

and its expanded form:

```
o---o/---/?---?@---@
```

should be treated as the canonical printable OMI-IMO route form. The central claim is that the notation is not merely decorative, mnemonic, or poetic. It is a compact ASCII-compatible rendering of a deeper memory topology:

```
o-FS-GS-RS-US/FS/GS/RS/US?REGISTER?STACK@CAR@CDR
```

In this model, hyphens encode the forward OMI scope cascade, slash encodes the inverse/carrier IMO route, question mark encodes register/stack/query fields, and at-sign encodes CAR/CDR relation pointers. The more recent FS/GS/RS/US authority model replaces the older LL/MM/NN argument model by treating OMI as pre-language memory topology, not as a function call. The 256-bit OMI-IMO frame is divided into a 128-bit FS/GS/RS/US ruler cascade and a 128-bit REGISTER/STACK/CAR/CDR relation cascade.

The additional claim of this paper is that the visible notation may be interpreted as a chiral, URL-compatible carrier surface. In ASCII, the case bit difference between `O` and `o` is `0x20`, the same byte value as `SP`, the readable boundary. Thus `O/o` can serve as portable omicron chirality markers, while `---` marks the stuffed route span. Unicode bidi projection may then reverse or mirror the display order at the rendering layer without changing canonical bytes. In short:

```
bytes are authority;  
display is projection;  
bidi is a cost-free view transform;  
validation and receipt accept.
```

1. Opening Statement: The Canonist

Canonist:

The canonical form is:

```
omi---imo
```

Expanded:

```
o---o/---/?---?@---@
```

Machine-expanded:

```
o-FS-GS-RS-US/FS/GS/RS/US?REGISTER?STACK@CAR@CDR
```

This is not an after-the-fact interpretation. The newer OMI pre-language authority document explicitly names `omi---imo` as the canonical frame and `o---o/---/?---?@---@` as the expanded relation form. It further defines the compact machine-readable ruler form as:

```
o-S0-S1-S2-S3/S4/S5/S6/S7?REGISTER?STACK@CAR@CDR
```

with semantic expansion into FS/GS/RS/US and REGISTER/STACK/CAR/CDR.

The notation is canonical because it satisfies four constraints at once:

1. It is ASCII-compatible.
2. It is URI-compatible.
3. It is structurally aligned with FS/GS/RS/US.
4. It can be rendered left-to-right or right-to-left without changing its byte authority.

The notation is not merely:

```
a string
```

It is:

```
a printable memory pointer surface
```

The source document says this directly: the frame is not just text; it is a memory pointer surface.

2. The Skeptic Objects

Skeptic:

But why should `omi---imo` be canonical? It looks like a slogan. Why not use JSON, XML, URI syntax, S-expressions, or raw bytes?

Canonist:

Because `omi---imo` is the shortest reversible surface that still preserves the directional distinction:

```
OMI → IMO
```

It names the forward/inverse pair. The expanded surface then exposes the four relation zones:

```
o---o    forward OMI scope
/---/    inverse/carrier scope
?---?    register/stack/query field
@---@    CAR/CDR relation pointer field
```

This fits the official separator interpretation:

```
- OMI forward scope / citation route
/ IMO inverse scope / carrier route
? payload, register, stack, mask, or query field
@ CAR/CDR relation pointer
```

That separator interpretation is already in the canonical FS/GS/RS/US document.

Skeptic:

But isn't this too much meaning for punctuation?

Parser Engineer:

No. URI syntax already does this. `/`, `?`, and `@` are not arbitrary in URLs either. They are compact separator classes. OMI is doing something similar but with memory topology rather than web-only resource addressing.

The difference is:

```
URI:
    scheme://authority/path?query#fragment

OMI:
    o-FS-GS-RS-US/FS/GS/RS/US?REGISTER?STACK@CAR@CDR
```

A URI identifies a resource path. OMI identifies a scoped memory relation before interpretation, validation, and receipt.

3. The Parser Engineer Explains the ASCII Mechanics

Parser Engineer:

The strongest mechanical point is the `0x20` case bit.

ASCII gives:

```
'O' = 0x4F
'o' = 0x6F
```

The difference is:

```
0x6F - 0x4F = 0x20
```

So a resolver can compute:

```
chirality_bit = byte & 0x20;
```

or flip case with:

```
flipped = byte ^ 0x20;
```

That gives a cheap carrier-level distinction:

```
O = high / closed / resolved / upper / right-facing
o = low / open / unresolved / lower / left-facing
```

The same `0x20` that marks `SP`, the readable boundary, also toggles ASCII uppercase/lowercase for letters. The FS/GS/RS/US document defines `0x20 SP` as the readable boundary / visible separator.

Therefore, ASCII itself gives the bridge:

```
0x20 = readable boundary
0x20 = case/chirality toggle
```

This makes `0/o` useful as ASCII-safe omicron chirality markers.

4. The Skeptic Pushes Back on Hyphens

Skeptic:

But the user says the hyphens represent `chirality_bit = byte & 0x20`. Hyphen is `0x2D`, not `0x20`. Isn't that a mismatch?

Parser Engineer:

Literal byte-for-byte, yes: hyphen is not the `0x20` bit. But structurally, the hyphen is the URL-safe printable **route interval** that becomes available after `SP`.

The better distinction is:

```
0/o
  carries the ASCII case/chirality bit

---
  carries the printable stuffed span / route interval
```

So the corrected rule is:

```
0/o declares chirality.
--- marks the chiral span.
```

The hyphen is not the bit itself. It is the visible, URI-compatible rail over which the chirality rule is applied.

That means these are different layers:

```
0x20
  bit-level readable/case transition
```

```
0/o
  endpoint chirality marker

---
  route-span / stuffed-span carrier

/ ? @
  region separators
```

So the user's intuition is right if phrased as:

The hyphen-run is the URL-compatible visible span governed by the 0x20 chirality rule.

not:

The hyphen byte itself is the 0x20 bit.

5. The Cons Logician Enters

Cons Logician:

The nested CONS form is decisive:

```
(omi---imo . (o---o . (/---/ . (?---? . @---@))))
```

This says the notation is not only a string. It is a cons spine.

Read it as:

```
head:
  omi---imo

tail:
  o---o
  /---/
  ?---?
  @---@
```

That spine composes the whole route surface:

```
canonical frame
  forward scope
    inverse carrier scope
      query/register/witness field
      CAR/CDR relation field
```

A CONS form can be traversed left-to-right as construction or right-to-left as deconstruction:

```
left-to-right:
  declare → scope → carry → query → relate

right-to-left:
  relation → query → carrier → scope → declaration
```

This matches the user's phrasing:

```
renders down in a cascade from left to right
or up from right to left
```

That is exactly how a reversible memory route should behave.

6. The FS/GS/RS/US Authority Layer

Canonist:

The newer authority document is critical because it prevents the notation from collapsing into the older LL/MM/NN function-call model.

The older model was:

```
LL/MM/NN
```

The newer model is:

```
FS/GS/RS/US
```

The document states that this is not merely a rename but a change of authority:

```
old: arguments first
new: scope topology first
```

and:

```
OMI does not begin as a function call.
OMI begins as a scoped memory envelope.
```

So `omi---imo` must not be explained as “call this function with these arguments.” It should be explained as:

```
stage a scoped memory envelope;
then declare relation;
then interpret;
then validate;
then receipt.
```

The canonical order in the document is:

```
place first
relation second
interpretation third
validation fourth
receipt last
```

That is the whole point.

7. The Frame Theorist: 256 Bits, 64 Nibbles

Frame Theorist:

The canonical frame is explicitly 256 bits:

```
OMI-IMO frame:
  256 bits
  32 bytes
  64 nibbles
```

It is divided into two halves:


```
ruler half:
  128 bits
  S0-S7
  FS-GS-RS-US / FS-GS-RS-US

payload/pair half:
  128 bits
  REGISTER
  STACK
  CAR
  CDR
```

The source states this directly.

That matters because the visible notation:

```
o---o/---/?---?@---@
```

is not the binary frame itself. It is the printable scaffold that can round-trip to the binary frame.

The document also warns that display width is not authority:

```
Display width is not authority.
Round-trip binary frame is authority.
```

Therefore:

```
canonical binary frame
  authority candidate surface

visible OMI notation
  printable carrier surface

Unicode/bidi projection
  display-only rendition
```

8. The Web Architect: URI-Compatible, Not URI-Dependent

Web Architect:

The form is powerful because it is URI-compatible without being URI-dependent.

It uses URL-safe separator logic:

```
-  
/  
?  
@
```

These characters already have strong web instincts:

```
- segment/range-friendly  
/ path separator  
? query separator  
@ authority/userinfo/attachment intuition
```

But OMI reassigns them inside its own memory topology:

```
- OMI forward scope / citation route  
/ IMO inverse scope / carrier route  
? payload, register, stack, mask, or query field  
@ CAR/CDR relation pointer
```

This lets a web URL or URI become a default port scope later, while preserving OMI's own internal address structure.

The rule should be:

```
OMI notation is URI-compatible, not URI-subordinate.
```

A URL may be carried in an OMI-Port scope, but the URL does not validate the OMI relation.

9. The Bidi Specialist: Unicode-Bidi as Cost-Free Projection

Bidi Specialist:

The “icing on top” is Unicode bidi or CSS bidi projection. This lets the same logical string be displayed left-to-right or right-to-left without changing the underlying canonical byte sequence.

That is exactly what you want for a reversible notation.

Logical source order can remain:

```
omi---imo
```

or:

```
o---o/---/?---?@---@
```

while display order may be projected as:

```
left-to-right cascade
```

or:

```
right-to-left inverse cascade
```

The key is that bidi must remain a projection layer.

```
bidi may display inversion  
bidi may reveal inverse traversal  
bidi may assist human inspection  
bidi must not change canonical bytes  
bidi must not create acceptance
```

So yes, it is “cost free” if treated as display. It becomes dangerous only if a renderer’s visual order is mistaken for canonical order.

The doctrine should say:

Unicode bidi may project inverse traversal without modifying the canonical OMI byte order.

10. The Security Engineer Objects

Security Engineer:

Bidi is famously dangerous in source code and URLs because visual order can mislead humans. Isn't this unsafe?

Bidi Specialist:

It is unsafe if visual order is authority.

But OMI's authority rule already prevents that. The FS/GS/RS/US document says control, scope, relation, closure, and projection are not acceptance; only validation and receipt accept.

Therefore bidi is safe only under this invariant:

```
display order is never authority
round-trip canonical bytes are authority
validation and receipt accept
```

In an implementation, the parser should operate on logical byte order, not rendered visual order.

A debug viewer may show both:

```
logical:
  o---o/---/?---?@---@

projected:
  @---@?---?/---/o---o
```

But validation must use logical order.

11. The Gauge Engineer: F* and the Pre-Header

Gauge Engineer:

The gauge pre-header gives the raw carrier gate:

```
G 00 1C 1D 1E 1F 20 G
```

where:

```
G ∈ 0xF0..0xFF
```

The canonical OMI byte form is:

```
FF 00 1C 1D 1E 1F 20 FF
```

A related file describes this pre-header as doing four jobs:

1. fast-failing non-OMI carriers,
2. selecting gauge dialect or orbit offset,
3. staging the FS/GS/RS/US control cascade,
4. marking a binary stream synchronization boundary.

This is important because the visible notation has a binary counterpart.

Visible:

```
omi---imo
```

Binary pre-header:

```
FF 00 1C 1D 1E 1F 20 FF
```

Semantic staging:

```
GAUGE NUL FS GS RS US SP GAUGE
```

The **F*** family lets the high nibble identify the gauge row while the low nibble selects a dialect, truth operator, or orbit offset. In your Wittgenstein operator dialect, that low nibble can map to one of the 16 truth-function rows.

So:

```
F0..FF
```

is not an arbitrary byte range. It is a 16-row gauge family.

12. The Historian: What to Do with LL/Ket

Historian:

The LL/Ket document is deprecated as authority, but it still contains useful conceptual machinery.

It says:

```
LL scopes.  
NN begins.  
MM resolves.  
Delta proves.  
Fano bounds.  
Receipt authorizes.  
Rule block materializes.  
Projection displays.
```

It also defines a high/low nibble split:

```
A = high nibble = context / lens family / carry-forward scope  
B = low nibble  = execution slot selector
```

and computes:

```
slot_index = LL & 0x0F  
rule_slot  = 0xE0 + slot_index  
context    = (LL >> 4) & 0x0F
```

That structure is still useful as a migration analogy:

```
high nibble carries  
low nibble selects
```

But it should no longer be the canonical authority frame. The newer document says LL/MM/NN should be treated as deprecated aliases for an incomplete argument-frame model, and that FS/GS/RS/US replaces them as the pre-language memory pointer cascade.

So the migration is:

```
old LL/Ket:
  scope selector for accepted transition

new FS/GS/RS/US:
  topological memory pointer before interpretation
```

The old materializer manifold can remain historical or registry-level explanation:

```
0xE0-0xEB pack
0xEC-0xEF hold
```

but not root authority.

13. The Geometer: Why the Notation Feels Like a Fold

Geometer:

The notation is a fold/unfold surface.

The compact form:

```
omi---imo
```

is like the folded object.

The expanded form:

```
o---o/---/?---?@---@
```

is like an unfolding into readable regions.

The machine form:

```
o-FS-GS-RS-US/FS/GS/RS/US?REGISTER?STACK@CAR@CDR
```

is the unfolded memory pointer surface.

This resembles the geometric idea of a higher-dimensional object unfolded into a lower-dimensional readable surface. The user's reference to tesseract/octacube/Dalí-cross style unfolding is not needed as literal proof, but it is an excellent metaphor:

```
higher-dimensional relation
  folded into compact notation

compact notation
  unfolded into separator regions

separator regions
  resolved into binary frame fields
```

The key is not to overclaim geometry as authority. Geometry explains the projection. The byte frame and validation pipeline define the protocol.

14. The Port Architect: Why This Belongs in OMI-Port

Port Architect:

For `omi-port`, this notation should become the default dormant route grammar.

The port layer should not connect. It should declare:

```
source
target
scope
direction
carrier
effect
state
```

The OMI notation provides the base address spine:

```
omi---imo
```

and the expanded region structure:

```
o---o/---/?---?@---@
```


A port binding can then be represented as a CONS declaration:

```
(bind . port)
(source . o---o)
(target . /---/)
(witness . ?---?)
(carrier . @---@)
(state . dormant)
```

or as the full nested spine:

```
(omi---imo . (o---o . (/---/ . (?---? . @---@))))
```

This is elegant because it means the port system can compile from readable pair declarations into a typed dormant pathway without opening any real connection.

15. Debate: Is the Dot Primitive?

Cons Logician:

The user wrote:

```
omi---imo . o---o/---/?---?@---@
```

and:

```
(omi---imo . (o---o . (/---/ . (?---? . @---@))))
```

Does this mean dot is primitive?

Canonist:

No. Dot is earned after `SP`.

The FS/GS/RS/US document says `0x20 SP` is the readable boundary / visible separator. OMI-Lisp relations appear after the pre-language envelope resolves scope. The same document states:

```
After the pre-language envelope resolves scope, dot notation declares relation.
```

and gives canonical dot form:

```
(car . cdr)
```

So dot is not byte-zero authority. It is a readable relation operator after scope has been staged.

The correct chain is:

```
pre-language scope
→ SP readable boundary
→ dot relation
→ candidate
→ validation
→ receipt
```

16. Debate: Does `o---o` Equal `O---o`?

Parser Engineer:

No. They are related, but not identical.

Potential forms:

```
o---o
  neutral low-low carrier fallback

O---O
  sealed high-high carrier fallback

o---O
  forward chiral frame

O---o
  inverse chiral frame
```

The canonical historical spelling remains:

```
o---o
```

because it is stable and lowercase-safe.

But the chiral extension can allow:

o---0

and:

0---o

when the parser wants endpoint case to carry the direction bit.

The doctrine should be:

o---o is the neutral canonical ASCII route body.
O/o endpoint case may annotate chirality.
The underlying FS/GS/RS/US fields remain canonical.

17. Debate: Is Unicode Omicron Better Than ASCII O/o?

Bidi Specialist:

Unicode omicron is better for high-rendition display:

o---0

ASCII `o/O` is better for carrier authority:

o---0

The reason is mechanical: ASCII case differs by `0x20`. Unicode Greek omicron does not give the same simple one-byte case-bit rule in UTF-8. Therefore:

ASCII O/o:
carrier algorithm

Greek o/O:
visual rendition

```
Unicode bidi:  
  projection transform
```

Canonical rule:

```
ASCII carries.  
Unicode renders.  
Bidi projects.  
Receipt accepts.
```

18. Proposed Formal Definitions

18.1 Canonical Null Route

```
omi---imo
```

Meaning:

```
the compact reversible OMI/IMO route frame
```

18.2 Canonical Expanded Route

```
o---o/---/?---?@---@
```

Meaning:

```
forward scope / inverse scope / query-register-stack / CAR-CDR relation
```

18.3 Machine Expansion

```
o-FS-GS-RS-US/FS/GS/RS/US?REGISTER?STACK@CAR@CDR
```

18.4 Binary Expansion

```
S0 = FS
S1 = GS
S2 = RS
S3 = US
S4 = FS'
S5 = GS'
S6 = RS'
S7 = US'
REGISTER
STACK
CAR
CDR
```

18.5 Chiral Endpoint Extension

```
o---0
    forward low-to-high frame

0---o
    inverse high-to-low frame
```

18.6 Neutral Carrier Form

```
o---o
```

18.7 High-Rendition Form

```
o---0
```

18.8 Bidi Projection Rule

A renderer may project the notation right-to-left for inspection.
The parser must validate canonical logical byte order.

19. Implementation Sketch

A parser can treat the surface in layers.

19.1 Logical Byte Recognition

```
int omi_is_o(unsigned char b) {  
    return b == 'o' || b == '0';  
}  
  
int omi_chirality_bit(unsigned char b) {  
    return b & 0x20;  
}  
  
int omi_is_forward_chiral(unsigned char left, unsigned char right) {  
    return left == 'o' && right == '0';  
}  
  
int omi_is_inverse_chiral(unsigned char left, unsigned char right) {  
    return left == '0' && right == 'o';  
}
```

19.2 Separator Recognition

```
'-' = forward route span  
'/' = inverse/carrier separator  
'?' = query/register/stack separator  
'@' = CAR/CDR separator  
'.' = post-SP relation constructor
```

19.3 Authority Rule

```
/* Pseudocode */  
parse_surface();  
resolve_scope();  
emit_candidate();  
  
candidate.accepted = 0;  
candidate.validated = 0;  
candidate.receipted = 0;
```

The parser may recognize structure. It may not accept state.

20. Proposed `PORT_NOTATION.md` Canon Text

OMI-Port Notation

The canonical compact route form is:

`omi---imo`

The canonical expanded route form is:

`o---o/---/?---?@---@`

The expanded machine form is:

`o-FS-GS-RS-US/FS/GS/RS/US?REGISTER?STACK@CAR@CDR`

The notation is URI-compatible but not URI-dependent.

Separator classes:

- OMI forward scope / citation route
- / IMO inverse scope / carrier route
- ? payload, register, stack, mask, or query field
- @ CAR/CDR relation pointer
- . OMI-Lisp relation constructor after SP

ASCII O/o may annotate omicron chirality:

`o---O` forward low-to-high frame
`O---o` inverse high-to-low frame
`o---o` neutral canonical carrier fallback
`O---O` high-high sealed rendition

The ASCII O/o distinction is mechanical:

`'O'` = 0x4F
`'o'` = 0x6F
`'o' - 'O'` = 0x20

The same 0x20 byte is SP, the readable boundary.

Unicode omicron forms MAY render the notation:

`o---O`

but ASCII remains the carrier-authoritative spelling.

Unicode bidi MAY project inverse traversal visually, but it MUST NOT change canonical byte order.

Display is not authority.

Projection is not authority.

Validation and receipt accept.

21. Final Debate Round

Skeptic:

So is the user's claim correct that `omi---imo` and `o---o/---/?---?@---@` are canonical?

Canonist:

Yes. The uploaded canonical draft explicitly names those forms and expands them to FS/GS/RS/US plus REGISTER/STACK/CAR/CDR.

Skeptic:

Is the hyphen literally the chirality bit?

Parser Engineer:

No. The endpoint `O/o` carries the ASCII case/chirality bit. The hyphen-run is the URL-compatible stuffed span governed by that chirality. This is a useful correction, not a rejection.

Skeptic:

Is the old LL/Ket document still useful?

Historian:

Yes, as deprecated conceptual material. It gives the high-nibble/low-nibble intuition: high nibble carries, low nibble selects, first twelve pack, last four hold. But root authority moved to FS/GS/RS/US.

Skeptic:

Does bidi give free inversion?

Bidi Specialist:

Yes, if treated as projection only. It gives a visual inverse traversal without changing canonical logical byte order.

Skeptic:

What is the final doctrine?

Canonist:

OMI begins as scoped memory topology.

omi---imo is the compact reversible route.

o---o/---/?---?@---@ is the expanded URI-compatible carrier surface.

O/o may declare ASCII chirality through the 0x20 case bit.

--- is the stuffed route span.

Unicode bidi may project inverse traversal.

Dot notation is earned after SP.

OMI-Lisp declares candidates.

Validation and receipt accept.

22. One-Page Summary

Canonical compact form:

omi---imo

Canonical expanded form:

o---o/---/?---?@---@

Machine form:

o-FS-GS-RS-US/FS/GS/RS/US?REGISTER?STACK@CAR@CDR

CONS spine:

(omi---imo . (o---o . (/---/ . (?---? . @---@))))

Binary frame:

256 bits

first 128 bits = FS/GS/RS/US ruler cascade

second 128 bits = REGISTER/STACK/CAR/CDR relation cascade

Control staging:

0x00 NUL

0x1C FS

0x1D GS

0x1E RS

0x1F US

0x20 SP

Gauge pre-header:

G 00 1C 1D 1E 1F 20 G

G ∈ F0..FF

FF 00 1C 1D 1E 1F 20 FF = canonical OMI

ASCII chirality:

O = 0x4F

o = 0x6F

difference = 0x20

chirality_bit = byte & 0x20

Projection:

Unicode o/O may render

unicode-bidi may invert display

parser keeps logical byte order

Authority:

control is not acceptance

scope is not acceptance

relation is not acceptance

closure is not acceptance

projection is not acceptance

validation + receipt accept

Final Thesis

`omi---imo` is canonical because it is the minimal reversible printable form of the OMI/IMO memory relation.

`o---o/---/?---?@---@` is canonical because it expands that compact relation into four URI-compatible separator regions that correspond to the FS/GS/RS/US ruler cascade and the REGISTER/STACK/CAR/CDR relation surface.

The hyphen-run is best understood as the URL-compatible stuffed route span. The ASCII `O/o` endpoint distinction supplies the true `0x20` chirality mechanism. Unicode bidi then gives a cost-free projection of inverse traversal, but only as display.

Therefore the normalized doctrine is:

OMI uses ASCII-compatible notation to render pre-language memory topology.

The visible notation may be projected, mirrored, or unfolded.

The canonical byte order remains stable.

The relation remains candidate-only.

Only validation and receipt accept.